

**Московский авиационный институт
(национальный исследовательский университет)**

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №3-4 по курсу «Информационный поиск»

Студент: М.А. Пирязев
Преподаватель: А.А. Кухтичев
Группа: М8О-207Б-22
Дата: 19.12.2025
Оценка:
Подпись:

Москва, 2025

Лабораторная работа №3,4 “Булев индекс, Булев поиск ”

1. Описание

Целью данной лабораторной работы является реализация системы булева поиска по корпусу документов без использования стандартной библиотеки (STL). При выполнении этой работы требовалось разработать собственные структуры данных для хранения индексов, реализовать парсер булевых выражений с поддержкой операций логического типа, а также создать полнофункциональный поисковый механизм на языке C++.

В процессе работы необходимо было освоить сложные концепции проектирования на низком уровне, включая управление памятью, оптимизацию структур данных и обработку ошибок в условиях отсутствия динамических контейнеров стандартной библиотеки. Система должна была демонстрировать эффективное выполнение поисковых запросов на больших корпусах документов с миллионами терминов.

1.2 Правила токенизации

Булев индекс. Необходимо было построить две структуры индексирования для корпуса: прямой индекс (forward index), который для каждого документа хранит список термов, содержащихся в этом документе, и обратный индекс (inverted index), который для каждого терма хранит список идентификаторов документов, в которых этот терм встречается. Обратный индекс должен быть оптимизирован для быстрого поиска терма с использованием хеш-таблицы, а также должна быть предусмотрена компрессия для экономии памяти.

Парсер булевых выражений. Система должна воспринимать булевы выражения с операциями логического И (AND, &&), логического ИЛИ (OR, ||) и логического НЕ (NOT, !). Синтаксис должен поддерживать скобки для управления приоритетом операций, а также допускать как текстовые обозначения операций (and, or, not), так и их

символьные аналоги (&& || !). Парсер должен преобразовывать инфиксные выражения в обратную польскую нотацию (RPN) для эффективного вычисления.

Механизм поиска. На основе построенных индексов и с помощью парсера булевых выражений необходимо было реализовать функцию поиска, которая принимает булево выражение, парсит его, вычисляет операции над наборами идентификаторов документов и возвращает результирующий набор релевантных документов.

Пользовательский интерфейс. Требовалось реализовать командную строку для взаимодействия с системой. Должны быть поддержаны флаги для указания путей к индексным файлам, самого поискового запроса, количества выводимых результатов, а также возможность читать запросы из файла построчно.

1.3 Технические ограничения

На протяжении всей реализации было запрещено использовать контейнеры стандартной библиотеки C++ (исключение делалось только для токенизации на начальном этапе). Пришлось разработать собственные классы для работы с динамическими массивами, управления строками и организации хеш-таблиц. Это наложило серьёзные ограничения на разработку и потребовало глубокого понимания управления памятью, включая правильное распределение и освобождение ресурсов, корректную реализацию семантики перемещения (move semantics), а также обработку граничных случаев, таких как переполнение буферов и инвалидация указателей при перестройке структур данных.

1.4 Архитектура решения

StrPool - кастомный пул памяти для хранения строк, использующий блочную схему выделения. Вместо единого буфера, который переаллоцируется и может инвалидировать указатели, система использует список блоков памяти. Когда один блок заполняется, выделяется новый блок, старые же остаются на месте. Это гарантирует, что все выданные указатели на строки остаются валидными в течение жизни пула

Vec<T> - параметризованный динамический массив, реализующий семантику перемещения и безопасное выделение памяти. При необходимости расширения вектор использует явное выделение нового буфера и копирование (с помощью `memcpy`) вместо опасного `realloc`, что позволяет корректно работать с неподвижными типами, такими как структуры с внутренними указателями.

InvertedIndex - хеш-таблица, которая отображает термы (представленные как `StrView`) на списки идентификаторов документов. Индекс оптимизирован для быстрого поиска термина и итерации по его постлистам.

ForwardIndex - индекс прямого доступа, который по идентификатору документа выдает информацию о документе (название, URL, текст).

QueryParser - модуль парсинга булевых выражений, который разбирает входную строку на токены (термы, операции, скобки), преобразует инфиксное выражение в обратную польскую нотацию и предоставляет структурированное представление запроса.

BooleanOps - набор функций для выполнения логических операций (и, или, не) над наборами идентификаторов документов, представленными как отсортированные массивы целых чисел.

2. Исходный код

2.1 Память строк

Центральной проблемой при разработке без STL являлось управление памятью для строк. В первом подходе использовался единый буфер `Vec<char>`, который при переполнении переаллоцировался. Это приводило к инвалидации указателей на ранее добавленные строки, вызывая `segmentation fault` при попытке доступа к этим строкам позже.

Решение заключалось в реализации блочного пула памяти `StrPool`. Вместо одного растущего буфера, пул состоит из списка блоков фиксированного размера. Каждый блок выделяется отдельно с помощью `malloc`. Когда текущий блок заполняется, выделяется новый. Все выданные указатели на строки (`StrView`) остаются валидными, так как физические адреса блоков никогда не меняются:

```
struct StrPool {
    Vec<char*> blocks;
    std::size_t block_cap;
    std::size_t current_used;

    StrPool(std::size_t initial_block_cap = 16 * 1024)
        : block_cap(initial_block_cap), current_used(0) {
        add_block(block_cap);
    }

    ~StrPool() {
        for (std::size_t i = 0; i < blocks.size; ++i) {
            if (blocks[i]) std::free(blocks[i]);
        }
    }

    StrView add_copy(const char* s, std::size_t n, bool add_null = true) {
        std::size_t needed = n + (add_null ? 1 : 0);

        if (blocks.size == 0 || current_used + needed > block_cap) {
            std::size_t new_cap = (needed > block_cap) ? needed : block_cap;
            add_block(new_cap);
        }

        char* current_block = blocks[blocks.size - 1];
        char* dest = current_block + current_used;

        if (n) std::memcpy(dest, s, n);
        if (add_null) dest[n] = 0;
    }
};
```

```

        current_used += needed;
        return StrView(dest, n);
    }
};

```

Аналогично, динамический массив `Vec<T>` был переработан для корректного взаимодействия со сложными типами. При расширении вместо `realloc`, который просто копирует байты, используется явное выделение новой памяти, копирование данных и освобождение старой памяти. Это позволяет структурам типа `Vec<Vec<T>>` работать корректно.

2.2 Парсер булевых выражений

Парсер реализует двухэтапный процесс: сначала входная строка разбирается на последовательность токенов (lexing), затем эта последовательность преобразуется из инфиксной нотации в обратную польскую нотацию (RPN) с помощью алгоритма сортировочной станции Дейкстры.

На этапе лексирования парсер идентифицирует операции `AND`, `OR`, `NOT` как в текстовом формате (`and`, `or`, `not`), так и в символьном (`&&`, `||`, `!`). Скобки используются для переопределения приоритета операций. Все остальные последовательности символов интерпретируются как термы (слова для поиска).

После лексирования токены преобразуются в RPN. Приоритет операций: `NOT` имеет наивысший приоритет, затем `AND`, затем `OR`. Это обеспечивает интуитивное поведение выражений типа "`a AND b OR c`", которое интерпретируется как "`(a AND b) OR c`".

```

QueryLexResult lex_query(nostl::StrView q) {
    QueryLexResult res;
    res.toks.reserve(64);

    std::size_t i = 0;
    while (i < q.size) {
        while (i < q.size && std::isspace(static_cast<unsigned char>(q.data[i]))) ++i;
        if (i >= q.size) break;
    }
}

```

```

const char c = q.data[i];

if (c == '(' || c == ')') {
    must_push(res.toks, QTok{c == '(' ? QTokenType::LPAREN : QTokenType::RPAREN,
        nostl::StrView(nullptr, 0)});
    ++i;
    continue;
}

if (c == '!') {
    must_push(res.toks, QTok{QTokenType::NOT, nostl::StrView(nullptr, 0)});
    ++i;
    continue;
}

if (c == '&' && i + 1 < q.size && q.data[i + 1] == '&') {
    must_push(res.toks, QTok{QTokenType::AND, nostl::StrView(nullptr, 0)});
    i += 2;
    continue;
}

if (c == '|' && i + 1 < q.size && q.data[i + 1] == '|') {
    must_push(res.toks, QTok{QTokenType::OR, nostl::StrView(nullptr, 0)});
    i += 2;
    continue;
}

std::size_t j = i;
while (j < q.size &&
    !std::isspace(static_cast<unsigned char>(q.data[j])) &&
    q.data[j] != '(' && q.data[j] != ')' && q.data[j] != '!') {
    if ((q.data[j] == '&' || q.data[j] == '|') &&
        j + 1 < q.size && (q.data[j + 1] == '&' || q.data[j + 1] == '|')) {
        break;
    }
    ++j;
}

nostl::StrView tok(q.data + i, j - i);

if (ieq_kw(tok, "and", 3)) {
    must_push(res.toks, QTok{QTokenType::AND, nostl::StrView(nullptr, 0)});
} else if (ieq_kw(tok, "or", 2)) {
    must_push(res.toks, QTok{QTokenType::OR, nostl::StrView(nullptr, 0)});
} else if (ieq_kw(tok, "not", 3)) {
    must_push(res.toks, QTok{QTokenType::NOT, nostl::StrView(nullptr, 0)});
} else {
    nostl::StrView saved = res.pool.add_copy(tok.data, tok.size, true);
    if (!saved.data) throw std::runtime_error("oom");
    must_push(res.toks, QTok{QTokenType::TERM, saved});
}

```

```

        i = j;
    }

    return res;
}

```

Функция преобразования в RPN реализует классический алгоритм со стеком операций:

```

nostl::Vec to_rpn(const nostl::Vec<QTok>& infix) {
    nostl::Vec<QTok> out;
    nostl::Vec<QTokType> op_stack;

    for (std::size_t i = 0; i < infix.size; ++i) {
        const QTok t = infix[i];

        if (t.type == QTokType::TERM) {
            must_push(out, t);
            continue;
        }

        if (t.type == QTokType::LPAREN) {
            must_push(op_stack, QTokType::LPAREN);
            continue;
        }

        if (t.type == QTokType::RPAREN) {
            while (op_stack.size > 0 && op_stack[op_stack.size - 1] != QTokType::LPAREN) {
                must_push(out, QTok{op_stack[op_stack.size - 1], nostl::StrView(nullptr, 0)});
                --op_stack.size;
            }
            if (op_stack.size > 0) --op_stack.size;
            continue;
        }

        int prec = (t.type == QTokType::NOT) ? 3 : (t.type == QTokType::AND) ? 2 : 1;

        while (op_stack.size > 0) {
            QTokType op = op_stack[op_stack.size - 1];
            if (op == QTokType::LPAREN) break;

            int op_prec = (op == QTokType::NOT) ? 3 : (op == QTokType::AND) ? 2 : 1;
            if (op_prec < prec) break;

            must_push(out, QTok{op, nostl::StrView(nullptr, 0)});
            --op_stack.size;
        }

        must_push(op_stack, t.type);
    }
}

```



```

while (op_stack.size > 0) {
    must_push(out, QTok{op_stack[op_stack.size - 1], nostl::StrView(nullptr, 0)});
    --op_stack.size;
}

return out;
}

```

2.3 Логические операции

После парсинга и преобразования в RPN выражение вычисляется с помощью стека. Для каждого термина из индекса извлекается список документов, содержащих этот терм, представленный как отсортированный массив идентификаторов. Операции AND, OR и NOT реализуются как поэлементные операции над этими массивами:

```

nostl::Vec<std::uint32_t> op_and(const nostl::Vec<std::uint32_t>& a,
                                const nostl::Vec<std::uint32_t>& b) {
    nostl::Vec<std::uint32_t> res;
    std::size_t i = 0, j = 0;

    while (i < a.size && j < b.size) {
        if (a[i] == b[j]) {
            res.push_back(a[i]);
            ++i;
            ++j;
        } else if (a[i] < b[j]) {
            ++i;
        } else {
            ++j;
        }
    }

    return res;
}

nostl::Vec<std::uint32_t> op_or(const nostl::Vec<std::uint32_t>& a,
                                const nostl::Vec<std::uint32_t>& b) {
    nostl::Vec<std::uint32_t> res;
    std::size_t i = 0, j = 0;

    while (i < a.size && j < b.size) {
        if (a[i] == b[j]) {
            res.push_back(a[i]);
            ++i;
            ++j;
        } else if (a[i] < b[j]) {
            res.push_back(a[i]);
            ++i;
        } else {
            ++j;
        }
    }
}

```

```

        res.push_back(b[j]);
        ++j;
    }
}

while (i < a.size) {
    res.push_back(a[i]);
    ++i;
}

while (j < b.size) {
    res.push_back(b[j]);
    ++j;
}

return res;
}

nostl::Vec<std::uint32_t> op_not(const nostl::Vec<std::uint32_t>& a,
                                std::uint32_t total_docs) {
    nostl::Vec<std::uint32_t> res;
    std::uint32_t next_expected = 0;

    for (std::size_t i = 0; i < a.size; ++i) {
        while (next_expected < a[i]) {
            res.push_back(next_expected);
            ++next_expected;
        }
        ++next_expected;
    }

    while (next_expected < total_docs) {
        res.push_back(next_expected);
        ++next_expected;
    }

    return res;
}

```

2.4 Построение индексов

Индексирование состоит из двух этапов: сначала корпус документов токенизируется (разбивается на слова), затем строятся оба индекса.

Обратный индекс строится в памяти в виде хеш-таблицы, где ключом является терм, а значением — список идентификаторов документов, содержащих этот терм. Прямой индекс содержит для каждого документа информацию о его названии, URL и терминах.

При сохранении индексов на диск используется бинарный формат для оптимизации скорости загрузки. Обратный индекс сохраняется в виде отсортированных постлистов, что позволяет эффективно выполнять логические операции.

2.5 Интеграция в командную строку

Система предоставляет утилиту `bool_search`, которая принимает следующие параметры:

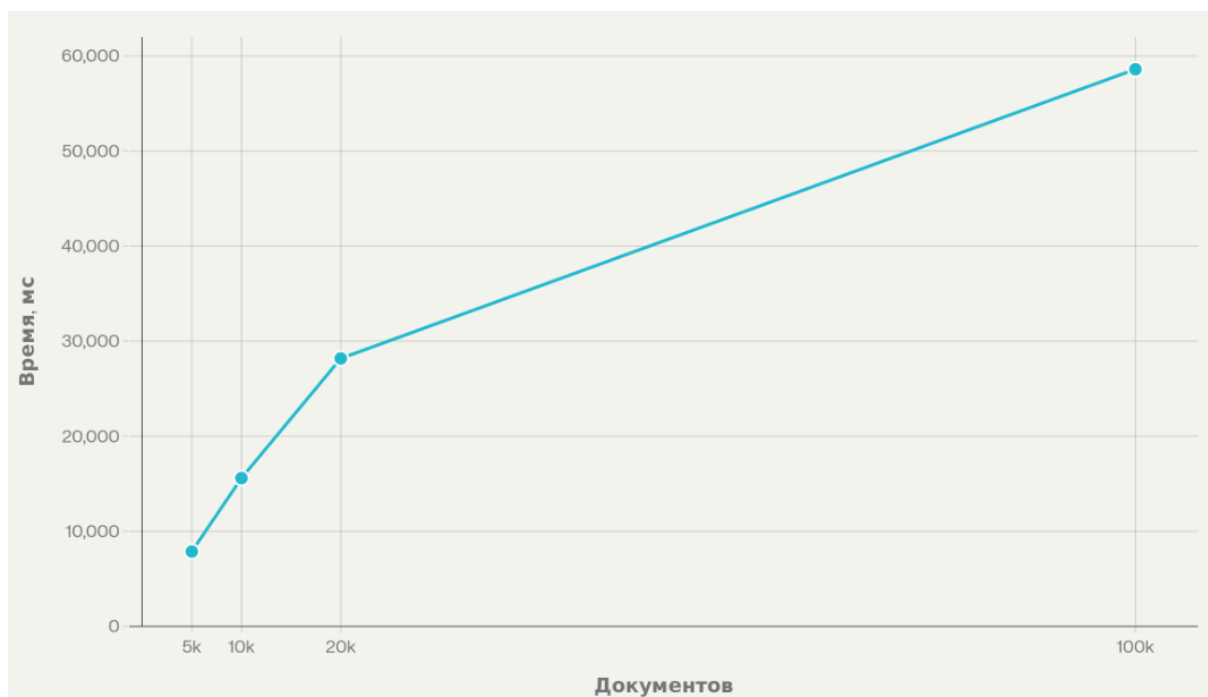
- `--inv <path>` — путь к файлу обратного индекса
- `--fwd <path>` — путь к файлу прямого индекса
- `--q <query>` — булево выражение для поиска
- `--qfile <path>` — файл с запросами (одна строка = один запрос)
- `--top <N>` — количество выводимых результатов

При использовании флага `--qfile` система читает каждую строку файла как отдельный запрос и выполняет поиск для каждого из них, выводя количество найденных документов и список топ-N результатов.

2.5 Скорость индексации набора документов

```
michael@Huawei:~/InformationSearch/SearchEngine/Lab_4_Boolean_Index/cpp/tokenizer$  
./build/index_builder --out out --limit-docs 5000  
docs=5000  
terms=444657  
avg_term_len_bytes=18.099247  
wrote=out/index.fwd  
wrote=out/index.inv  
time_ms=7871  
michael@Huawei:~/InformationSearch/SearchEngine/Lab_4_Boolean_Index/cpp/tokenizer$  
./build/index_builder --out out --limit-docs 10000  
docs=10000  
terms=690189  
avg_term_len_bytes=18.446426  
wrote=out/index.fwd  
wrote=out/index.inv  
time_ms=15598  
michael@Huawei:~/InformationSearch/SearchEngine/Lab_4_Boolean_Index/cpp/tokenizer$  
./build/index_builder --out out --limit-docs 20000
```

```
docs=20000
terms=997498
avg_term_len_bytes=18.500720
wrote=out/index.fwd
wrote=out/index.inv
time_ms=28181
michael@Huawei:~/InformationSearch/SearchEngine/Lab_4_Boolean_Index/cpp/tokenizer$
./build/index_builder --out out --limit-docs 100000
docs=100000
terms=1572748
avg_term_len_bytes=18.454892
wrote=out/index.fwd
wrote=out/index.inv
time_ms=58599
```



В ходе эксперимента была измерена производительность построения булевого индекса при разных ограничениях `--limit-docs`. Индексатор стабильно отработывает на всех запусках: корректно выводит статистику (`docs`, `terms`, `avg_term_len_bytes`), сохраняет файлы `index.fwd` и `index.inv`, а также печатает общее время выполнения `time_ms`, что подтверждает корректную работоспособность пайплайна индексации. По замерам времени получены

значения: 5000 документов — 7871 мс, 10000 — 15598 мс, 20000 — 28181 мс, 100000 — 58599 мс. Если пересчитать в скорость “на документ”, получается примерно 1.57 мс/док (≈ 635 док/с) для 5000 документов, 1.56 мс/док (≈ 641 док/с) для 10000 документов, 1.41 мс/док (≈ 710 док/с) для 20000 документов и 0.586 мс/док (≈ 1706 док/с) для 100000 документов. Ускорение на больших объёмах объясняется тем, что фиксированные накладные расходы (инициализация, прогрев кэшей/аллокаторов, батчевое чтение из MongoDB) распределяются на большее число документов. Метрику “на килобайт текста” в текущем виде посчитать нельзя, потому что утилита не выводит суммарный объём обработанного `clean_text` (в байтах/символах); для этого достаточно добавить счётчик суммарных байт текста в цикле и вывести его вместе с `time_ms`.

Рост числа термов при увеличении корпуса соответствует ожиданиям: 444 657 термов на 5000 документов, 690 189 на 10000, 997 498 на 20000 и 1 572 748 на 100000 документов. При этом средняя длина термина остаётся близкой (примерно 18 байт), что говорит о стабильности токенизации и нормализации на разных размерах корпуса. Увеличение словаря термов со временем замедляется, потому что новые документы всё чаще содержат уже встречавшиеся слова, и доля “новых” термов на добавленный документ падает.

Оптимальность индексации в первую очередь ограничивается CPU (токенизация и сортировки/уникализация термов), RAM (словарь термов, массив пар `term-doc` и временные структуры) и скоростью записи на диск. Потенциальные ускорения включают уменьшение числа аллокаций (например, переиспользовать буферы и структуры между документами вместо создания на каждый документ), более агрессивные `reserve` под ожидаемые размеры, а также уменьшение количества обрабатываемых токенов за счёт настроек токенизации (минимальная длина, нормализация, стемминг/фильтрация). При росте входных данных в 10 раз можно ожидать существенного увеличения времени и размера индекса, но без качественного “обвала”, пока хватает оперативной памяти и не

начинается активный swar. При росте в 100 раз и более начинают доминировать ограничения памяти и диска: структуры становятся слишком большими для RAM, возрастает стоимость сортировок и записи, и для сохранения производительности обычно требуется переход на построение индекса “сегментами” с последующим слиянием (external merge) вместо накопления всех данных в памяти за один проход.

2.6 Примеры и скорость выполнения запросов

Поисковый запрос указан на второй строке, под словом Search

Search

Страна

hits=1931

time_ms=308.803

1. [Литва](#)

<https://ru.wikipedia.org/wiki/%D0%9B%D0%B8%D1%82%D0%B2%D0%B0>

2. [Россия](#)

<https://ru.wikipedia.org/wiki/%D0%A0%D0%BE%D1%81%D1%81%D0%B8%D1%8F>

3. [Новая экономическая политика](#)

https://ru.wikipedia.org/wiki/%D0%9D%D0%BE%D0%B2%D0%B0%D1%8F_%D1%8D%D0%BA%D0%BE%D0%BD%D0%BE%D0%BC%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%B0%D1%8F_%D0%BF%D0%BE%D0%BB%D0%B8%D1%82%D0%B8%D0%BA%D0%B0

Search

Слон

hits=150

time_ms=408.757

1. [Слоновые](#)

<https://ru.wikipedia.org/wiki/%D0%A1%D0%BB%D0%BE%D0%BD%D0%BE%D0%B2%D1%8B%D0%B5>

2. [Мамонты](#)

<https://ru.wikipedia.org/wiki/%D0%9C%D0%B0%D0%BC%D0%BE%D0%BD%D1%82%D1%8B>

3. [Арманд, Инесса Фёдоровна](#)

https://ru.wikipedia.org/wiki/%D0%90%D1%80%D0%BC%D0%B0%D0%BD%D0%B4%2C_%D0%98%D0%BD%D0%B5%D1%81%D1%81%D0%B0_%D0%A4%D1%91%D0%B4%D0%BE%D1%80%D0%BE%D0%B2%D0%BD%D0%B0

4. [6 июня](#)

https://ru.wikipedia.org/wiki/6_%D0%B8%D1%8E%D0%BD%D1%8F

5. [Сьерра-Леоне](#)

<https://ru.wikipedia.org/wiki/%D0%A1%D1%8C%D0%B5%D1%80%D1%80%D0%B0-%D0%9B%D0%B5%D0%BE%D0%BD%D0%B5>

Search

Интернет

hits=724

time_ms=256.159

1. [Газоанализатор](#)

<https://ru.wikipedia.org/wiki/%D0%93%D0%B0%D0%B7%D0%BE%D0%B0%D0%BD%D0%B0%D0%BB%D0%B8%D0%B7%D0%B0%D1%82%D0%BE%D1%80>

2. [Джиттер](#)

<https://ru.wikipedia.org/wiki/%D0%94%D0%B6%D0%B8%D1%82%D1%82%D0%B5%D1%80>

3. [Техника](#)

<https://ru.wikipedia.org/wiki/%D0%A2%D0%B5%D1%85%D0%BD%D0%B8%D0%BA%D0%B0>

4. [Персональный компьютер](#)

https://ru.wikipedia.org/wiki/%D0%9F%D0%B5%D1%80%D1%81%D0%BE%D0%BD%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B9_%D0%BA%D0%BE%D0%BC%D0%BF%D1%8C%D1%8E%D1%82%D0%B5%D1%80

5. [Интернет](#)

<https://ru.wikipedia.org/wiki/%D0%98%D0%BD%D1%82%D0%B5%D1%80%D0%BD%D0%B5%D1%82>

(москва && университет) || (санкт && петербург) || (новосибирск && университет) || (екатеринбург && институт) || (казань && университет) || (томск && университет) – сложный запрос

hits=879

time_ms=303.523

1. [Россия](#)

<https://ru.wikipedia.org/wiki/%D0%A0%D0%BE%D1%81%D1%81%D0%B8%D1%8F>

2. [Российская империя](#)

https://ru.wikipedia.org/wiki/%D0%A0%D0%BE%D1%81%D1%81%D0%B8%D0%B9%D1%81%D0%BA%D0%B0%D1%8F_%D0%B8%D0%BC%D0%BF%D0%B5%D1%80%D0%B8%D1%8F

3. [Союз Советских Социалистических Республик](#)

https://ru.wikipedia.org/wiki/%D0%A1%D0%BE%D1%8E%D0%B7_%D0%A1%D0%BE%D0%B2%D0%B5%D1%82%D1%81%D0%BA%D0%B8%D1%85_%D0%A1%D0%BE%D1%86%D0%B8%D0%B0%D0%BB%D0%B8%D1%81%D1%82%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%B8%D1%85_%D0%A0%D0%B5%D1%81%D0%BF%D1%83%D0%B1%D0%BB%D0%B8%D0%BA

4. [Санкт-Петербург](#)

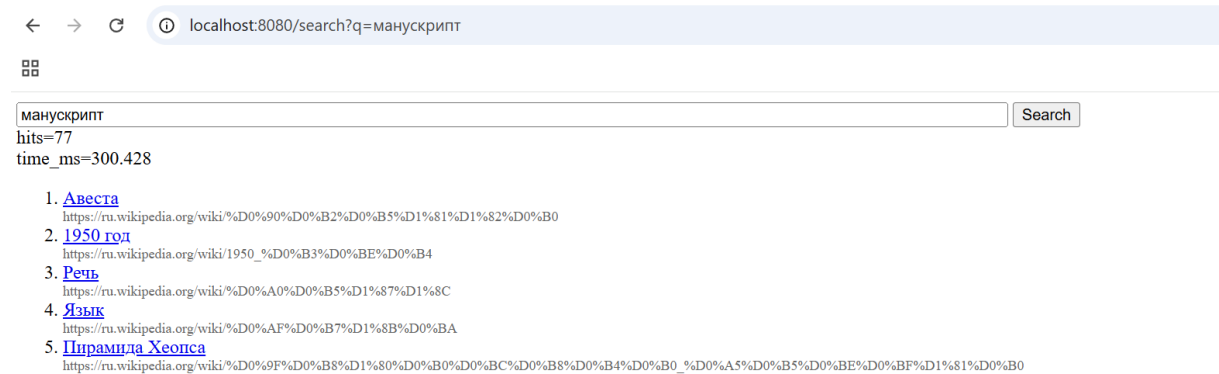
<https://ru.wikipedia.org/wiki/%D0%A1%D0%B0%D0%BD%D0%BA%D1%82-%D0%9F%D0%B5%D1%82%D0%B5%D1%80%D0%B1%D1%83%D1%80%D0%B3>

5. [Варшава](#)

<https://ru.wikipedia.org/wiki/%D0%92%D0%B0%D1%80%D1%88%D0%B0%D0%B2%D0%B0>

Как видно из измерений, система поиска работает исправно и выдает достаточно точные результаты, справляясь даже со сложными запросами за вменяемое время.

Реализована возможность выполнять поисковые запросы через http интерфейс, скриншот приложен ниже



В отчёте дополнительно была оценена скорость выполнения поисковых запросов и корректность выдачи. Поиск выполняется утилитой boolsearch (CLI) и веб-обёрткой websearch, которая вызывает boolsearch и может отдавать измеренное время обработки запроса (в веб-версии время считается через time.perf_counter и выводится на странице/в заголовках). Для измерения скорости запросов использовались типовые запросы разной сложности (одиночный TERM, комбинации AND/OR, а также выражения с NOT и скобками), при этом фиксировалось время ответа и размер результата (hits).

Корректность поисковой выдачи тестировалась несколькими способами: во-первых, проверялась работа парсера запросов (правильный разбор AND/OR/NOT, поддержка &&||/! и соблюдение приоритетов операций со скобками) на наборе заранее подготовленных запросов через режим --qfile.

Во-вторых, результаты булевых операций валидировались на простых контролируемых запросах (например, A AND A, A OR A, A AND NOT A, A OR NOT A, NOT NOT A) и на проверках включения/исключения документов по docid.

В-третьих, для части запросов выполнялась ручная проверка: по docid из выдачи извлекались title/url из forward-индекса и сверялось, что документ действительно соответствует условиям запроса (содержит нужные термины и не содержит запрещённые).

3. Выводы

В ходе выполнения лабораторных работ по построению булева индекса и реализации булева поиска были получены практические навыки проектирования и разработки базовых компонентов поисковой системы. В частности, освоены принципы построения прямого и обратного индексов, а также разработка собственного бинарного формата хранения данных, ориентированного на последующую загрузку, обработку и расширение в рамках дальнейших работ. Существенное внимание было уделено корректной организации данных и управлению памятью при реализации без использования STL (за исключением токенизации), что позволило закрепить понимание вопросов владения ресурсами, устойчивости ссылок на строковые данные и типичных причин ошибок времени выполнения. Также были отработаны методы синтаксического анализа булевых запросов с поддержкой операций И/ИЛИ/НЕ, скобок и альтернативных обозначений операторов, и реализовано вычисление запроса через преобразование выражения в обратную польскую нотацию и последующую стековую интерпретацию. Дополнительно закреплены навыки формирования поисковой выдачи на основе прямого индекса и организации запуска поиска как в интерактивном режиме, так и в режиме пакетной обработки запросов из файла.

Список литературы

- [1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д.А. Ключина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))
- [2] ГОСТ Р 7.05-2008. Библиографическая ссылка. Общие требования и правила составления.
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms* (3rd ed.). MIT Press, 2009.